Laborationsrapport 1

Laboration 1
DT171G D2990, DT171G, Inbäddade system för Internet of Things

Författare: Siye Abera och Omar Sultanzoy, siab2400@student.miun.se och omsu2400@student.miun.se
Termin, år: VT26, 2026.

Mittuniversitetet
MID SWEDEN UNIVERSITY

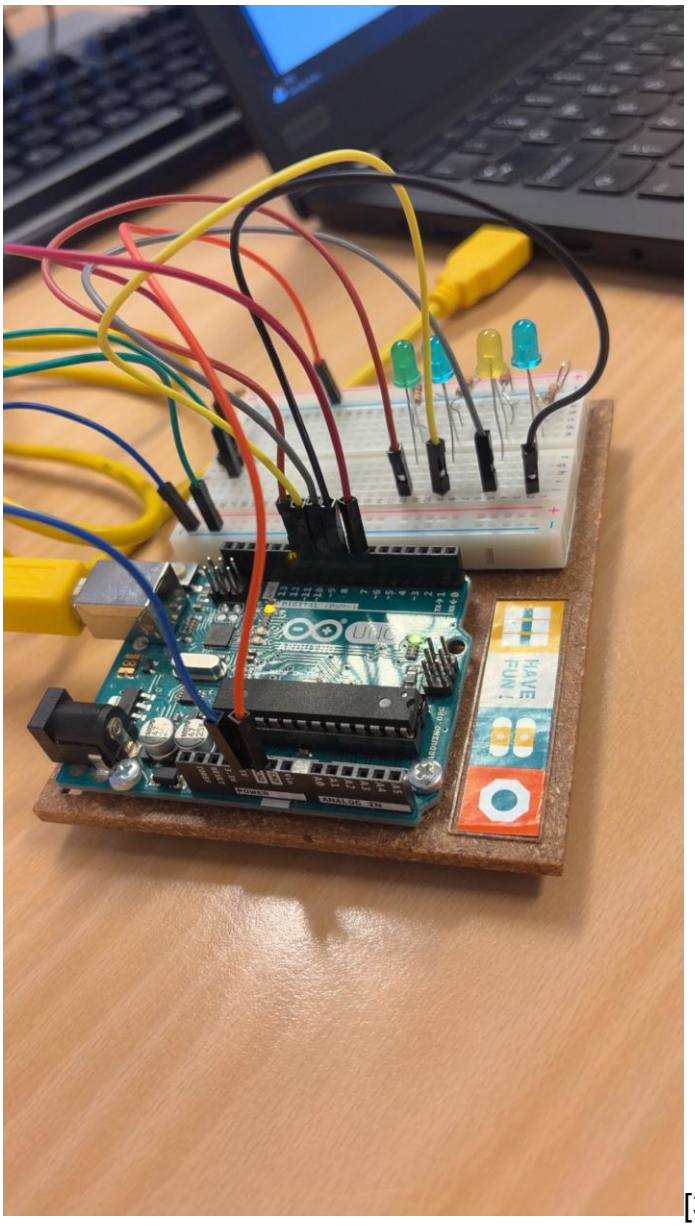
1 Sammanfattning

Genom att kombinera enkel hårdvara med grundläggande programmering i Arduino kunde ett binärt räknarsystem implementeras. Lösningen visar tydligt hur decimala tal kan översättas till binär form och visualiseras med lysdioder. Uppgiften gav god förståelse för digitala insignaler, utgångar, loopar och villkorssatser. I samband med koden, så skapades två flödesscheman som vi inkluderat i rapporten.

2 Innehållsförteckning

..... 1
1 Sammanfattning 2
2 Innehållsförteckning 3
3 Slutsatser 11
4 Källförteckning 12

Konstruktion



[3]

Del1:

Uppgiften gick ut på att med hjälp av en Arduino, fyra lysdioder och en tryckknapp visa binära tal från 0 till 15. Varje lysdiod representerar ett binärt värde (1, 2, 4 och 8). När användaren trycker på knappen ska Arduino räkna upp talen sekventiellt och visa motsvarande binära representation med lysdioderna.

Kretsen byggdes med en Arduino Uno, ett kopplingsdäck, fyra lysdioder med tillhörande motstånd samt en tryckknapp. Lysdioderna kopplades till digitala utgångar på Arduino (pin 8–11) och tryckknappen till pin 7. Arduino matas via USB från datorn.

Varje variabel representerar en bit och tilldelas den pin där motsvarande lysdiod är kopplad:

```
*/
const int bitOne = 11;
const int bitTwo = 10;
const int bitFour = 9;
const int bitEight = 8;
const int buttonPin = 7;
```
[3]

Namnet representerar vilken bit det är, och värdet som tilldelas är vilken pin som den är kopplad till på arduinon.

Här initisialiseras lysdioderna som utgångar och knappen som ingång:

```
void setup() {
  pinMode(bitOne, OUTPUT);
  pinMode(bitTwo, OUTPUT);
  pinMode(bitFour, OUTPUT);
  pinMode(bitEight, OUTPUT);
  pinMode(buttonPin, INPUT);
}
```
[3]

Programmet väntar tills knappen trycks in. När detta sker startar en for-loop som räknar från 0 till 15. För varje varv analyseras värdet och bryts ned i binära delar genom att jämföra mot 8, 4, 2 och 1. Om värdet är större än eller lika med bitens vikt tänds motsvarande lysdiod. Vi har delay på 1000ms mellan varje lysdiod.

```
void loop() {
  if (digitalRead(buttonPin) == HIGH) { // När jag klickar på knappen, gör
som står nedan.
    for (int count = 0; count <= 15; count++) { // For loop som itererar 16
gånger, 0-15.
      int temp = count;

      if (temp >= 8) { // Bit 4 (längst till vänster, värde 8)
        digitalWrite(bitEight, HIGH); // Om sant, lys den lampan
        temp -= 8; // Subtrahera värdet från temp
      }
      else {
        digitalWrite(bitEight, LOW); // Annars håll den avstängd
      }

      if (temp >= 4) { // Bit 3 (värde 4)
        digitalWrite(bitFour, HIGH);
        temp -= 4;
      }
      else {
        digitalWrite(bitFour, LOW);
      }

      if (temp >= 2) { // Bit 3 (värde 2)
        digitalWrite(bitTwo, HIGH);
```

[3]

Vi släcker lamporna efteråt:

```
      /*
      Släcker alla lampor
      */
      digitalWrite(bitEight, LOW);
      digitalWrite(bitFour, LOW);
      digitalWrite(bitTwo, LOW);
      digitalWrite(bitOne, LOW);
    }
}
```

[3]

Del 2:

I del 2 vidareutvecklades lösningen från del 1 genom att lägga till möjligheten att räkna både uppåt och nedåt mellan 0 och 15. Detta uppnås med hjälp av en tryckknapp som växlar räknarens riktning.

En variabel count används för att lagra det aktuella talet, medan variabeln r anger riktningen på räknaren. När r = 1 räknar systemet uppåt och när r = -1 räknar

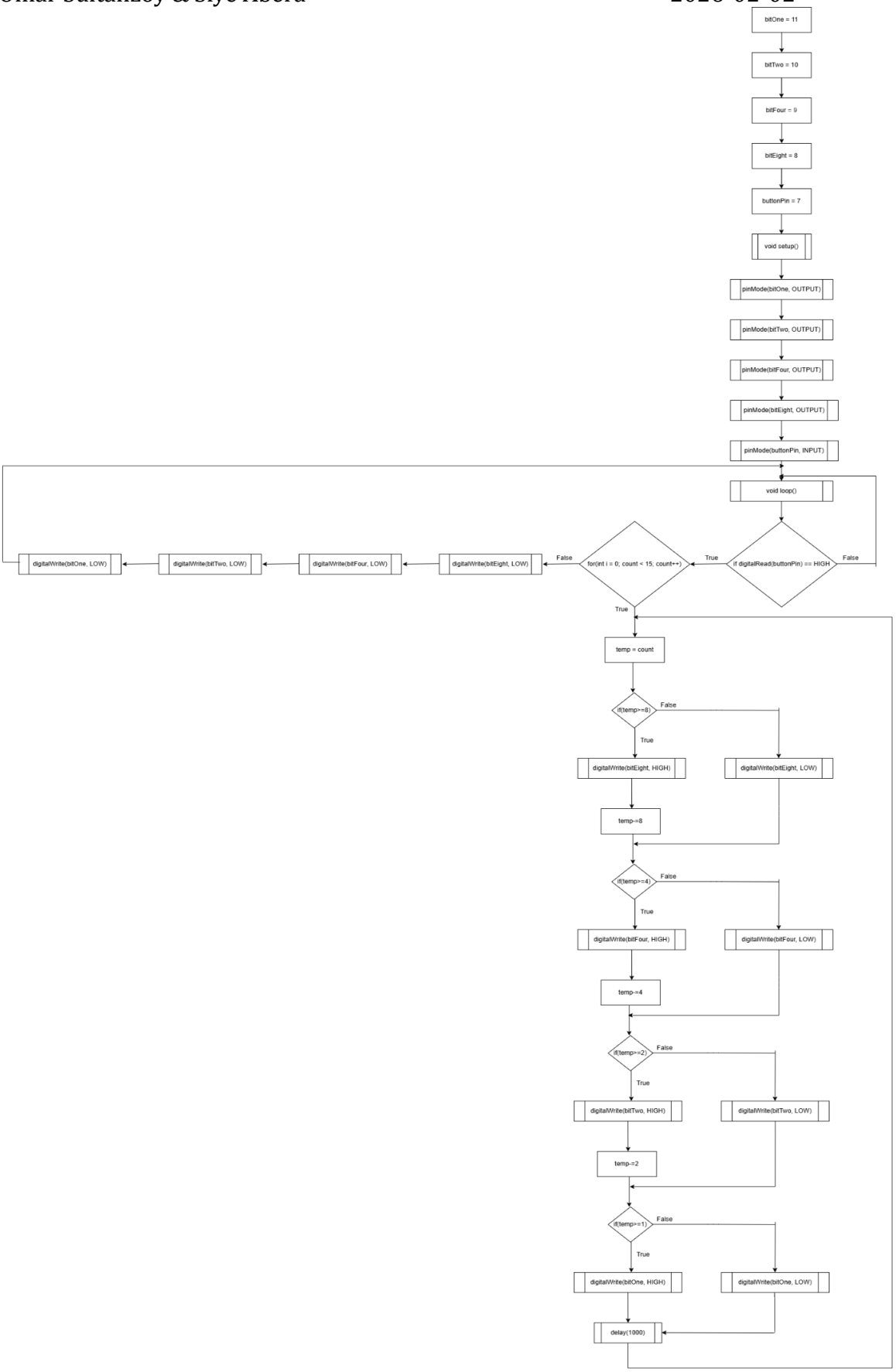
systemet nedåt. Varje gång knappen trycks in ändras riktningen genom att värdet på r byts mellan 1 och -1.

För att undvika att ett knapptryck registreras flera gånger används en kort fördröjning delay(300), vilket fungerar som enkel debouncing. Räknaren uppdateras sedan varje sekund och begränsas till intervallet 0–15 genom att värdet nollställs vid överskridning av gränserna.

Det aktuella decimala värdet konverteras därefter till binär form genom att jämföra talet mot bitvärdena 8, 4, 2 och 1. Motsvarande lysdioder tänds eller släcks, vilket ger en visuell representation av talet i binärt format. För felsökning och verifiering skrivs även räknarens värde ut i den seriella monitorn.

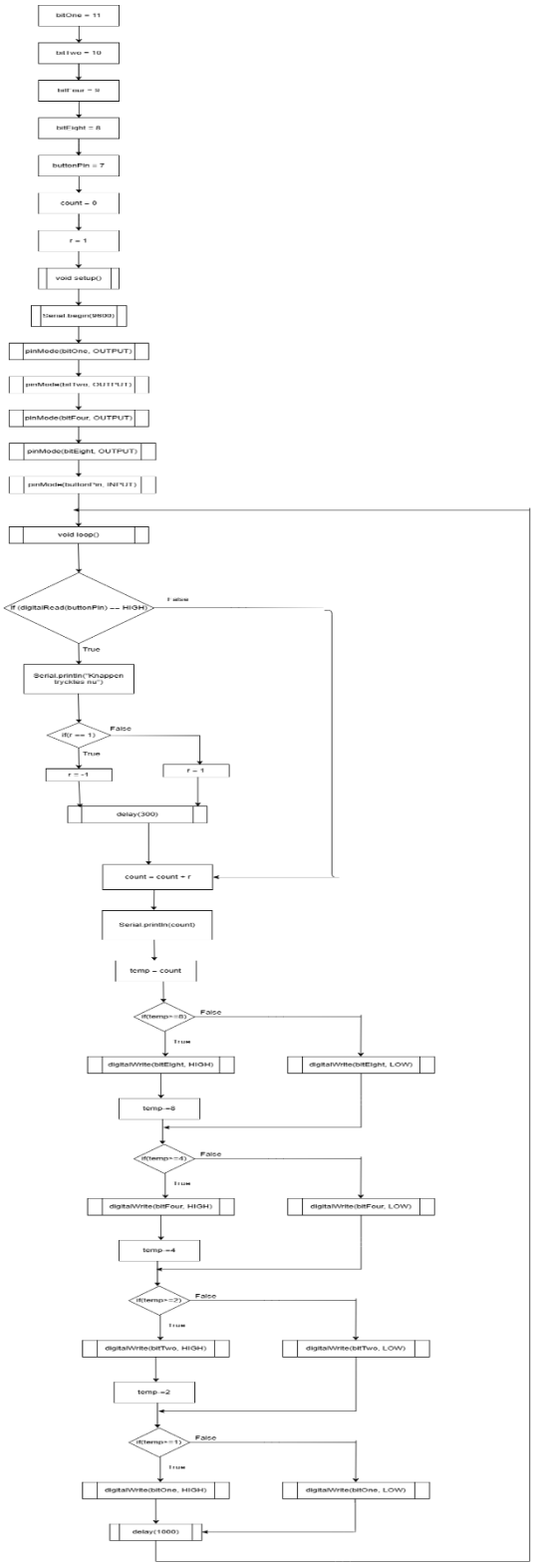
Vi upplevde problem i början med att när vi track på knappen så gick inte det baklängest pga vi hade inte en debuggning kod, efteråt så gick det bra. Serial.println hjälpt och en delay på 300 ms för att ge koden tid för känna igen när vi pressade på knappen. Det lede till lösningen av vårt problem.

```
void loop() {
  // Kolla om knappen trycks för att byta riktning
  if (digitalRead(buttonPin) == HIGH) {
    Serial.println("Knappen trycktes nu");
    if (r == 1) {
      r = -1;
    } else {
      r = 1;
    }
    delay(300); // Hindrar att den växlar 100 ggr på ett tryck
  }

  count = count + r;

  if (count > 15) {
    count = 0;
  }
  if (count < 0) {
    count = 15;
  }

  Serial.println(count);

  int temp = count;
```
[3]



[3]

Figur 1: Flödesschema



[3]

Figur 2: Flödesschema

Figur 1 visar flödesschemat för del 1 av laborationen. I flödesschemat ser man hur koden fungerar, vad som händer och när allt händer samt hur koden är inkapslad. I flödesschemat visas hur programmet väntar på en insignal från knappen för att sedan gå in i en for-loop. I for-loopen ser man hur algoritmen steg för steg körs.

Figur 2 visar exakt som flödesschema 1 fast för källkoden i del 2. Inte så stor skillnad på koden, men några ändringar är gjord och flödesschemat visar skillnaden. Skillnaden är att for-loopen ersätts av en riktningsvariabel som styr.

Båda flödesscheman är skapad med en extension i Visual Studio Code som heter ”drawio”.

3 Slutsatser

Av uppgiften har vi lärt oss hur binära tal fungerar i praktiken och hur de kan visualiseras med hjälp av lysdioder och en Arduino. Genom att bygga och programmera kretsen blev det tydligt hur varje bit påverkar det slutliga värdet, vilket gjorde den teoretiska delen lättare att förstå.

Uppgiften gav också en bättre förståelse för hur programmering och hårdvara samverkar. Genom att använda en knapp för att ändra räknarens riktning såg vi hur användarinput direkt kan påverka programmets beteende. Vi fick även insikt i hur viktigt det är att hantera gränsvärden och knapptryck på rätt sätt för att undvika oväntade fel.

Sammanfattningsvis var uppgiften lärorik och gav en bra grund inom både digitalteknik och Arduino-programmering, samtidigt som den visade hur relativt enkla komponenter kan användas för att skapa ett fungerande och tydligt system.

4 Källförteckning

[1] Youtube, “How to use a Breadboard ” https://www.youtube.com/watch?v=6WReFkfrUIk Publicerad 2015-09-17. Hämtad 2026-02-02.

[2] Youtube, ”Arduino UNO Tutorial #1 - Intro to Arduino w/TinkerCAD Circuits”, https://www.youtube.com/watch?v=3kDMYomFw5o Publicerad 2018-01-30. Hämtad 2005-02-02.

[3] Omar Sultanzoy & Siye Abera ,”Screenshot”. Tagen 2026-02-02. Hämtad 2026-02-02.